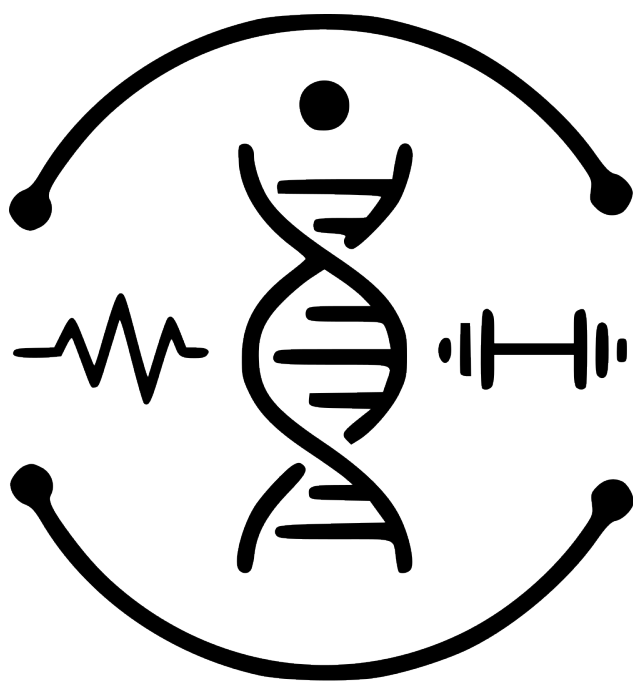


Migraineaid High-Level Technical Specification

S. M. Aqdas Rizvi

May 28, 2025



AIHI

1 Introduction

This document outlines the technical specification for the rework of an existing Android Java migraine tracking application, MigraineAid. The new platform will be developed using React Native for the client application and Rust for backend services. Key goals include enhancing functionality, improving the data schema, enabling machine learning for migraine prediction, and ensuring robust data privacy and security by design, adhering to relevant regulations (e.g., HIPAA, GDPR). The platform will support patients in tracking their migraines, allow consented data sharing with doctors, and enable consented data access for researchers via a dedicated dashboard.

2 Guiding Principles

- **Patient-Centricity:** The platform must be intuitive and genuinely beneficial for patients.
- **Privacy & Security by Design:** Data protection is a foundational element, not an afterthought.
- **Explicit User Consent:** All data processing and sharing will be based on granular, explicit user consent.
- **Scalability & Maintainability:** The architecture will support growth in users and data, and facilitate ongoing development.

3 System Architecture

The platform will consist of the following major components:

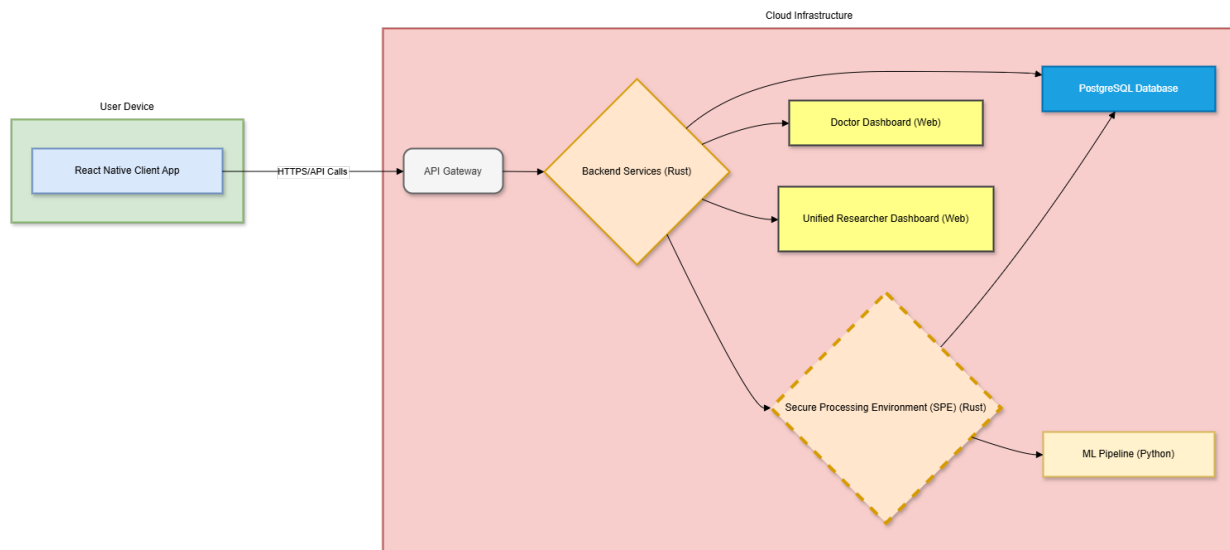


Figure 1: Overall System Architecture

- **React Native Client App:** The primary interface for patients. Responsible for data input, offline storage, client-side encryption/decryption, and synchronization.

- **API Gateway:** Single entry point for client requests, handling authentication, rate limiting, and routing.
- **Backend Services (Rust):** Handles core business logic, user management, consent management, orchestration of data to/from the database and SPE. Rust is chosen for its performance, memory safety, and strong concurrency features, crucial for handling sensitive data and ensuring system reliability.
- **PostgreSQL Database:** The primary data store for user accounts, encrypted health data, consent records, and other application data. PostgreSQL offers robustness, ACID compliance, and powerful querying capabilities.
- **Secure Processing Environment (SPE) (Rust):** A highly isolated and secure backend component, also built in Rust. It is the *only* component authorized to temporarily decrypt user data for specific, consented server-side operations. For user-initiated requests (e.g., personalized ML predictions), it uses client-derived, short-lived Session Encryption Keys (SEKs) based on the user's DEK. For system-initiated batch processing (e.g., preparing anonymized datasets for research), the SPE has a highly secure, audited mechanism to access and use user DEKs temporarily, strictly within its isolated environment. In all cases, plaintext DEKs and sensitive data are discarded by the SPE immediately after the specific operation is complete. Future iterations of the platform will investigate the integration of Fully Homomorphic Encryption (FHE) capabilities within or alongside the SPE. The objective would be to perform certain sensitive computations (e.g., aspects of machine learning inference, privacy-preserving federated learning updates, or statistical aggregations for research) directly on FHE-encrypted data. This would eliminate the need for even temporary decryption of the underlying data by server-side components for these specific operations, offering an unparalleled level of privacy. Libraries such as OpenFHE (currently usable via bindings for Rust for the underlying C/C++ code) provide a pathway for this exploration. A long-term research goal could involve contributing to or fostering the development of a native Rust FHE library, utilizing Rust's safety and performance characteristics to build highly secure and efficient homomorphic computation engines tailored for mHealth applications.
- **ML Pipeline (Python):** Dedicated services for training machine learning models (e.g., for migraine prediction) and serving predictions. Python is chosen for its extensive data science and ML ecosystem.
- **Doctor Dashboard (Web):** A web application allowing verified doctors to view data of their specific patients who have explicitly consented to share.
- **Unified Researcher Dashboard (Web):** A single web portal for verified researchers to access data according to project approvals and user consent levels (anonymized, pseudonymized, or, in highly controlled scenarios, identifiable).

4 Data Schema & Encryption

A relational schema will be implemented in PostgreSQL. Sensitive Personal Health Information (PHI) will be protected using client-side encryption.

- **Client-Side Encryption (CSE):**

- **Data Encryption Key (DEK):** A unique AES-256 (or similar strong symmetric cipher) key per user, generated on the client. This key encrypts the user's sensitive health data fields before they are transmitted to or stored on the server.
- **Master Encryption Key (MEK):** Derived on the client from the user's password using a strong key derivation function (e.g., Argon2, PBKDF2). The MEK encrypts the DEK.
- **encrypted_DEK:** The DEK, encrypted by the MEK, is stored in the **Users** table on the server. The server never has access to the plaintext MEK or DEK.
- **Local Client Storage:** The React Native app will use an encrypted local database (e.g., SQLite with SQLCipher), with the database encryption key being the user's plaintext DEK (held in memory after login).
- **Encrypted Fields:** Specific fields within tables (e.g., `DailyEntries.notes_encrypted`, `MigraineEvents.intensity_encrypted`) will store the ciphertext. Non-sensitive metadata or fields required for server-side indexing/querying (e.g., `user_id`, `entry_date`) will not be client-side encrypted.

Diagram: Encryption Key Hierarchy

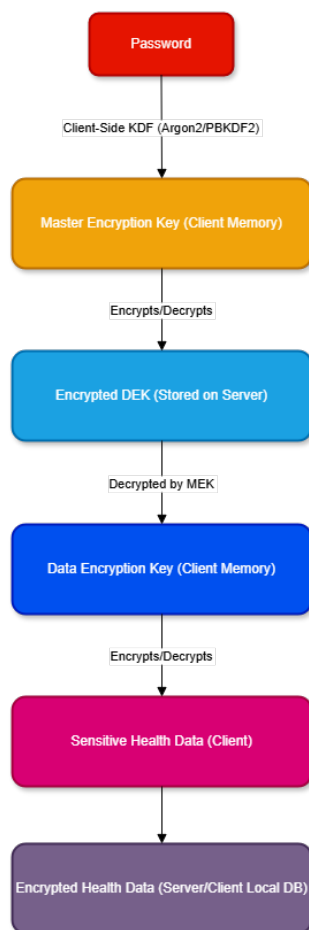


Figure 2: Encryption Key Hierarchy

5 Key Management & User Lifecycle

- **Registration:**

1. User provides email and sets a password.
2. Client generates DEK, derives MEK from password, encrypts DEK with MEK to create **encrypted_DEK**.
3. Client generates a unique **Recovery Code**. The user is instructed to store this securely offline. This code is the sole method for account/data recovery if the password is forgotten.
4. Client sends **password_hash** (server-side hash of password) and **encrypted_DEK** to the server.

- **Login:**

1. User enters password. Client derives MEK.
2. Client fetches **encrypted_DEK** from the server.
3. Client decrypts **encrypted_DEK** using MEK to obtain the plaintext DEK in memory.
4. DEK is used to decrypt/encrypt data for display/sync and to unlock the local encrypted database.

- **Password/Data Recovery:**

1. User indicates "Forgot Password."
2. Client prompts for the user-held Recovery Code.
3. If valid, the Recovery Code is used by the client to decrypt the **encrypted_DEK** (or an alternate DEK encrypted by a key derived from the recovery code).
4. User sets a new password, MEK is re-derived, DEK is re-encrypted with the new MEK, and the updated **encrypted_DEK** and new **password_hash** are sent to the server.

- **New Device / Data Migration:**

1. User logs in on the new device.
2. Plaintext DEK is obtained as per the login flow.
3. Client performs a full sync of encrypted data from the server.
4. Data is decrypted client-side using the DEK and stored in the new device's local encrypted database.

6 Consent Management

- **ConsentDefinitions Table:** Stores definitions of all consent types (e.g., Terms of Service, Privacy Policy, Personalized ML Predictions, Anonymized Data for General Research, Pseudonymized Data for Specific Research Project, Identifiable Data for Specific Research Project, Sharing Data with Specific Doctor). Includes versioning and plain language summaries.

- **ConsentRecords Table:** Links Users to ConsentDefinitions, recording `is_granted` status, timestamps, and `scope_details` (JSONB for specifics like `doctor_id` or `research_project_id`).
- **Mechanism:** Users will have a dedicated section in the app to view and manage their consents granularly. All data processing and sharing by backend systems (especially the SPE) will be gated by checks against these consent records.

7 Data Flow & Processing for Specific Use Cases

- **Patient Data Entry & Viewing:** All encryption/decryption for the patient's own viewing occurs on the client device using their DEK.
- **Personalized ML Predictions:**
 - Requires explicit user consent.
 - The client derives a short-lived Session Encryption Key (SEK) from the user's DEK (e.g., using HKDF with appropriate context). This SEK is securely provided to the SPE on a per-request basis for this specific operation.
 - The SPE uses the provided SEK to decrypt necessary data, passes it to the Python ML service for inference, receives the prediction, and then discards the SEK and plaintext data.
- **Doctor Access:**
 - **Prerequisites:** Verified doctor account with a registered public/private key pair (public key stored on server); explicit patient consent to share with this specific doctor.
 - **Sharing Setup (One-time per consent):**
 1. Patient's app (holding patient's DEK) fetches the doctor's public key from the server.
 2. Patient's app encrypts the patient's DEK with the doctor's public key, creating an `encrypted_patient_dek_for_doctor`.
 3. This `encrypted_patient_dek_for_doctor` is sent to the server and stored (e.g., in `PatientDoctorRelationships`), linked to the consent.
 - **Doctor Viewing Data:**
 1. Doctor logs into their dashboard; their client application loads their private key.
 2. Upon selecting a consented patient, the doctor's dashboard client fetches the relevant `encrypted_patient_dek_for_doctor` and the patient's encrypted health data blobs from the server (after server-side authorization check).
 3. The doctor's client uses the doctor's private key to decrypt `encrypted_patient_dek_for_doctor`, yielding the patient's plaintext DEK in the doctor's client memory.
 4. The doctor's client uses this patient DEK to decrypt and display the patient's health data.
 - **Revocation of Access:** When the patient revokes consent, the server marks the consent and relationship as inactive and, crucially, deletes or invalidates the `encrypted_patient_dek_for_doctor`. This prevents the doctor from obtaining the patient's DEK for future sessions. Doctor dashboards should not persistently store patient DEKs.

- **Researcher Access: Anonymized/Pseudonymized Data (via SPE Batch Processing)**
 - Requires explicit user consent for a specific research project requiring anonymized/pseudonymized data.
 - **Data Preparation (System-Initiated Batch Process):**
 1. The backend system identifies users who have consented to share data with the specific research project.
 2. For each consenting user, the SPE:
 - * Securely obtains the user’s DEK (e.g., by decrypting the user’s `encrypted_DEK` using a highly protected system-level key within its isolated environment). This mechanism is strictly for system-level operations.
 - * Uses the DEK to decrypt the consented raw data fields.
 - * Applies rigorous anonymization or pseudonymization techniques (e.g., k-anonymity, differential privacy, replacing direct identifiers with project-specific pseudonyms).
 - * Securely discards the plaintext DEK and raw decrypted data for that user immediately after processing.
 3. The SPE compiles the processed data from all consenting users into an anonymized/pseudonymized dataset for the research project.
 - **Researcher Accessing Processed Dataset:**
 1. Authenticated researcher accesses the Unified Researcher Dashboard.
 2. The dashboard presents the pre-prepared anonymized/pseudonymized dataset for their approved projects.
 3. The researcher interacts with this already processed dataset; no further decryption involving user DEKs occurs at this stage.
 - **Revocation of Consent:** If a user revokes consent, their data will not be included in *future* generations of the anonymized/pseudonymized dataset. Policies will define handling for data already incorporated into such datasets.

Researcher Access: Identifiable/Direct Pseudonymized Data (SPE-Mediated, Highly Controlled)

- Requires explicit user consent for a specific research project that has approval for this level of data access.
- **Data Access (On-Demand or Prepared View via SPE):**
 1. Authenticated researcher requests access to specific consented patient data (or a consented cohort) for their approved project via the Researcher Dashboard.
 2. The backend verifies consent and orchestrates the SPE.
 3. The SPE securely obtains the relevant user(s)’ DEK(s) (as described in the batch processing above).
 4. The SPE uses the DEK(s) to decrypt the specific consented data fields.
 5. The SPE may apply project-specific pseudonymization if required (e.g., consistent patient pseudonyms for the study).

6. The SPE provides this prepared (identifiable or direct pseudonymized) data subset to a secure API consumed by the Researcher Dashboard. Plaintext DEKs and raw data are discarded by SPE.
- **Researcher Viewing Data:** The Researcher Dashboard displays the SPE-provided data within its controlled environment. Data export is typically heavily restricted or disabled. All access is meticulously audited.
 - **Revocation of Consent:** If a user revokes consent, the backend immediately blocks any further requests from the researcher (for that project) to access that user’s data via the SPE.

Diagram: Consented Data Access via SPE

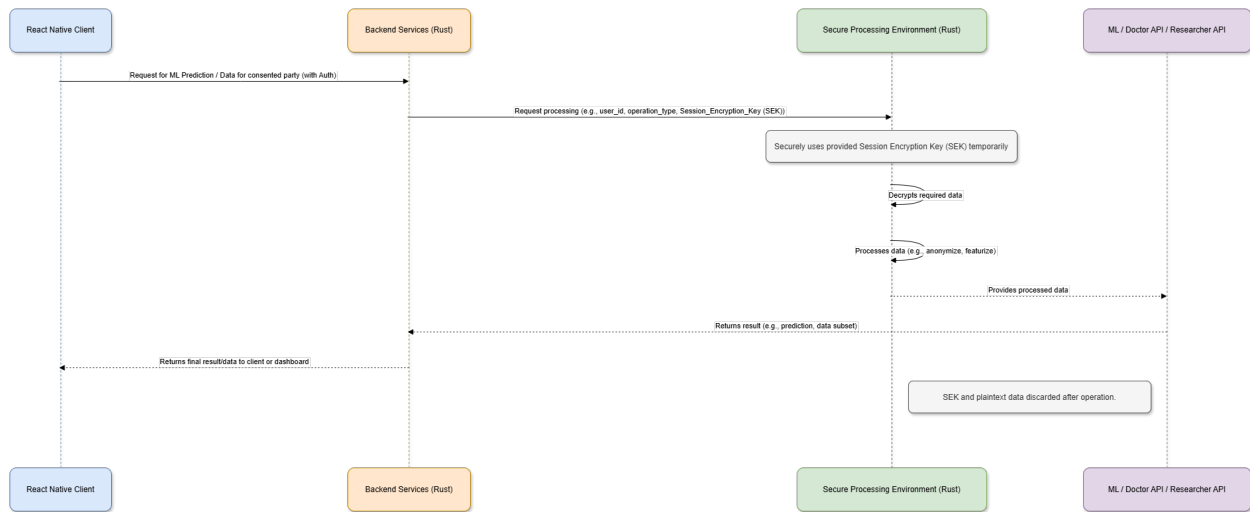


Figure 3: Consented Data Access via SPE

8 Offline Support & Synchronization

- The React Native app will support offline data entry. Data will be stored in the local encrypted SQLite database.
- A synchronization mechanism (e.g., batch sync of pending records, or change data capture) will be implemented to send locally stored encrypted data to the backend when network connectivity is available.
- The backend API will handle batch uploads and provide status updates for synced records.

9 Security Considerations (Summary)

- **End-to-End Principle for Sensitive Data:** Sensitive data is encrypted on the client before transmission and stored encrypted at rest on the server. It is only decrypted on the client for the user or temporarily within the SPE for consented server-side operations.
- **Key Derivation:** Strong KDFs for MEK.
- **Recovery Mechanism:** User-controlled recovery code.
- **Secure SPE:** Hardened, isolated component with minimal privileges.
- **Authentication & Authorization:** Robust mechanisms for all API access.
- **Input Validation:** Both client-side and server-side.
- **Audit Trails:** Comprehensive logging of sensitive operations, especially consent changes and data access events via SPE.

- **Regular Security Reviews & Testing:** Penetration testing and code reviews.
- **Derived Session Keys for SPE:** Utilizing derived, short-lived session keys for operations within the Secure Processing Environment (SPE) to minimize exposure of the primary Data Encryption Key (DEK).

10 Future Considerations

The following points are noted for potential future development phases but are out of scope for the initial rework:

- Advanced ML model explainability (XAI).
- Direct integration with wearable sensor data beyond basic aggregates.
- Federated learning approaches (if feasible and beneficial for ML).
- Investigate and prototype the use of FHE (e.g., via OpenFHE with Rust bindings) for specific server-side computations to further enhance privacy. This includes exploring FHE-compatible machine learning models and data aggregation techniques.
- Evaluate the performance trade-offs and usability implications of integrating FHE into the platform.
- Explore the research potential and feasibility of developing or contributing to a native Rust FHE library, which would align with the platform's core backend technology and Rust's strengths in security and performance. This could significantly advance the state-of-the-art in privacy-preserving mHealth solutions.

A Database Schema Information

This appendix details the proposed relational database schema for the PostgreSQL database. Fields intended for client-side encryption are suffixed with `_encrypted` and will typically be stored as `BYTEA` (byte array) or encrypted `TEXT`. Non-sensitive fields or those required for server-side filtering without decryption will be stored in their native types. Primary Keys (PK) are typically UUIDs to ensure global uniqueness, especially useful with client-generated data. Foreign Keys (FK) establish relationships between tables. Timestamps are stored with time zone information (`TIMESTAMP WITH TIMEZONE`).

A.1 Users Table

Stores information about registered users, including their encrypted data encryption key.

- `user_id` (UUID, PK)
- `email_encrypted` (BYTEA, client-side encrypted email, if not used as primary login identifier)
- `username` (VARCHAR, UNIQUE, if a non-email login identifier is used)
- `password_hash` (VARCHAR, server-side hash of the user's password)
- `encrypted_dek` (BYTEA, user's Data Encryption Key, encrypted with their MEK)
- `recovery_code_data_hashed` (VARCHAR, NULLABLE, server-side hash of data related to the recovery code for verification purposes. This would typically be a hash of the recovery code itself, used only to verify its correctness during the recovery process without storing the code in plaintext. This field and its handling require extreme security scrutiny.)
- `profile_data_encrypted` (BYTEA, e.g., name, year of birth, gender)
- `preferences` (JSONB, non-sensitive user preferences like app theme, notification settings)
- `created_at` (TIMESTAMP WITH TIME ZONE)
- `updated_at` (TIMESTAMP WITH TIME ZONE)

A.2 ConsentDefinitions Table

Master table for all types of consent offered within the platform.

- `consent_definition_id` (VARCHAR(100), PK, e.g., `TERMS_V1.0`, `RESEARCH_ANON_V1.0`)
- `title` (VARCHAR(255), NOT NULL)
- `description` (TEXT, NOT NULL)
- `version` (VARCHAR(20), NOT NULL)
- `category` (VARCHAR(50), NOT NULL, e.g., `MANDATORY_LEGAL`, `DATA_USAGE_RESEARCH`)
- `user_facing_summary` (TEXT, NULLABLE)
- `is_active` (BOOLEAN, DEFAULT true)
- `created_at` (TIMESTAMP WITH TIME ZONE)

A.3 ConsentRecords Table

Tracks individual user consents based on the definitions.

- `consent_record_id` (UUID, PK)
- `user_id` (UUID, FK to `Users.user_id`, NOT NULL)
- `consent_definition_id` (VARCHAR(100), FK to `ConsentDefinitions.consent_definition_id`, NOT NULL)
- `is_granted` (BOOLEAN, NOT NULL)
- `granted_at` (TIMESTAMP WITH TIME ZONE, NULLABLE)
- `revoked_at` (TIMESTAMP WITH TIME ZONE, NULLABLE)
- `scope_details` (JSONB, NULLABLE, e.g., `\{"doctor_id":"uuid","research_project_id":"uuid"\}`)
- `last_updated_at` (TIMESTAMP WITH TIME ZONE)
- *Constraint: UNIQUE (user_id, consent_definition_id)*

A.4 DailyEntries Table

Tracks daily patient-reported information, some of which is sensitive and encrypted.

- `entry_id` (UUID, PK)
- `user_id` (UUID, FK to `Users.user_id`)
- `entry_date` (DATE, The specific day this log is for. Not encrypted for querying.)
- `created_at_client` (TIMESTAMP WITH TIME ZONE, Timestamp from client)
- `sleep_quality_encrypted` (BYTEA, NULLABLE)
- `stress_level_encrypted` (BYTEA, NULLABLE)
- `mood_level_encrypted` (BYTEA, NULLABLE)
- `energy_level_encrypted` (BYTEA, NULLABLE)
- `menstruation_status_encrypted` (BYTEA, NULLABLE)
- `weather_conditions_encrypted` (BYTEA, NULLABLE, e.g., notes on perceived weather impact)
- `diet_notes_encrypted` (BYTEA, NULLABLE)
- `water_intake_ml_encrypted` (BYTEA, NULLABLE)
- `activity_level_category` (VARCHAR, NULLABLE, e.g., `SEDENTARY`, `LIGHT` - non-sensitive category for basic filtering)
- `notes_encrypted` (BYTEA, NULLABLE, general daily notes)

- `created_at` (TIMESTAMP WITH TIME ZONE, Server timestamp)
- `updated_at` (TIMESTAMP WITH TIME ZONE, Server timestamp)
- *Constraint: UNIQUE (user_id, entry_date)*

A.5 MigraineEvents Table

Logs details of individual migraine attacks.

- `event_id` (UUID, PK)
- `user_id` (UUID, FK to Users.user_id)
- `daily_entry_id` (UUID, FK to DailyEntries.entry_id, NULLABLE)
- `start_timestamp` (TIMESTAMP WITH TIME ZONE, Not encrypted for temporal querying)
- `end_timestamp` (TIMESTAMP WITH TIME ZONE, NULLABLE, Not encrypted)
- `intensity_encrypted` (BYTEA, NULLABLE)
- `pain_location_encrypted` (BYTEA, NULLABLE)
- `has_aura` (BOOLEAN, NULLABLE, Non-sensitive flag for basic filtering)
- `aura_description_encrypted` (BYTEA, NULLABLE)
- `impact_on_activities_encrypted` (BYTEA, NULLABLE)
- `notes_encrypted` (BYTEA, NULLABLE, specific notes about the migraine event)
- `created_at_client` (TIMESTAMP WITH TIME ZONE)
- `created_at` (TIMESTAMP WITH TIME ZONE)
- `updated_at` (TIMESTAMP WITH TIME ZONE)

A.6 Symptoms Table (Master List)

A predefined list of common migraine symptoms. Not user-specific or encrypted.

- `symptom_id` (SERIAL, PK)
- `symptom_name` (VARCHAR, UNIQUE)
- `symptom_category` (VARCHAR, NULLABLE)

A.7 EventSymptoms Table (Junction)

Links migraine events to experienced symptoms.

- `event_symptom_id` (UUID, PK)
- `event_id` (UUID, FK to `MigraineEvents.event_id`)
- `symptom_id` (INTEGER, FK to `Symptoms.symptom_id`)
- `severity_encrypted` (BYTEA, NULLABLE)
- `details_encrypted` (BYTEA, NULLABLE, e.g., specific nature of the symptom)
- *Constraint: UNIQUE (event_id, symptom_id)*

A.8 Triggers Table (Master List)

A predefined list of common migraine triggers. Not user-specific or encrypted.

- `trigger_id` (SERIAL, PK)
- `trigger_name` (VARCHAR, UNIQUE)
- `trigger_category` (VARCHAR, NULLABLE)

A.9 EventTriggers Table (Junction)

Links migraine events to perceived triggers.

- `event_trigger_id` (UUID, PK)
- `event_id` (UUID, FK to `MigraineEvents.event_id`)
- `trigger_id` (INTEGER, FK to `Triggers.trigger_id`)
- `user_confidence_encrypted` (BYTEA, NULLABLE, "How sure are you this was a trigger?")
- *Constraint: UNIQUE (event_id, trigger_id)*

A.10 Helpers Table (Master List)

A predefined list of common wellness practices or coping mechanisms (helpers) that users might employ. Not user-specific or encrypted.

- `helper\_id` (SERIAL, PK)
- `helper\_name` (VARCHAR, UNIQUE, e.g., "Meditation", "Hydration", "Dark Room", "Reduced Screen Time")
- `helper\_category` (VARCHAR, NULLABLE, e.g., "Mindfulness", "Lifestyle", "Comfort Measure")
- `description` (TEXT, NULLABLE)

A.11 DailyLogHelpers Table (Junction)

Links daily log entries to helpers used or noted by the patient on that day. This allows tracking of proactive measures or general wellness activities.

- `daily\_log\_helper\_id` (UUID, PK)
- `daily\_entry\_id` (UUID, FK to `DailyEntries.entry\_id`)
- `helper\_id` (INTEGER, FK to `Helpers.helper\_id`)
- `effectiveness\_encrypted` (BYTEA, NULLABLE, User's perceived effectiveness, e.g., 1-5 scale)
- `details\_encrypted` (BYTEA, NULLABLE, e.g., "Meditated for 15 minutes in the morning")
- `timestamp\_logged\_client` (TIMESTAMP WITH TIME ZONE, When the helper was logged by the client)
- *Constraint: UNIQUE (daily_entry_id, helper_id)*

A.12 Medications Table (Master List)

A predefined list of medications and treatments. Not user-specific or encrypted.

- `medication_id` (SERIAL, PK)
- `medication_name` (VARCHAR, UNIQUE)
- `medication_type` (VARCHAR, e.g., PRESCRIPTION, OTC, LIFESTYLE)

A.13 EventTreatments Table (Junction)

Logs treatments used during a migraine event.

- `event_treatment_id` (UUID, PK)
- `event_id` (UUID, FK to `MigraineEvents.event_id`)
- `medication_id` (INTEGER, FK to `Medications.medication_id`)
- `timestamp_taken_encrypted` (BYTEA)
- `dosage_encrypted` (BYTEA, NULLABLE)
- `effectiveness_encrypted` (BYTEA, NULLABLE)
- `side_effects_encrypted` (BYTEA, NULLABLE)

A.14 SensorDataAggregates Table

Stores aggregated sensor data from the client device, if collected.

- `aggregate_id` (UUID, PK)
- `user_id` (UUID, FK to `Users.user_id`)
- `daily_entry_id` (UUID, FK to `DailyEntries.entry_id`, NULLABLE)
- `data_type` (VARCHAR, e.g., `AVG_LUMINANCE`, `STEP_COUNT`)
- `value_encrypted` (BYTEA, the aggregated sensor value)
- `start_time_period` (TIMESTAMP WITH TIME ZONE)
- `end_time_period` (TIMESTAMP WITH TIME ZONE)
- `recorded_at` (TIMESTAMP WITH TIME ZONE)

A.15 Doctors Table

Stores information about verified doctors who can access patient data (with consent).

- `doctor_id` (UUID, PK)
- `email` (VARCHAR, UNIQUE)
- `full_name_encrypted` (BYTEA)
- `public_key` (TEXT, Doctor's public key for DEK sharing)
- `verification_status` (VARCHAR, e.g., `PENDING`, `VERIFIED`)
- `specialty_encrypted` (BYTEA, NULLABLE)
- `created_at` (TIMESTAMP WITH TIME ZONE)

A.16 PatientDoctorRelationships Table

Manages consent-based data sharing relationships between patients and doctors.

- `relationship_id` (UUID, PK)
- `patient_user_id` (UUID, FK to `Users.user_id`)
- `doctor_id` (UUID, FK to `Doctors.doctor_id`)
- `consent_record_id` (UUID, FK to `ConsentRecords.consent_record_id`, specific consent for this sharing)
- `encrypted_patient_dek_for_doctor` (BYTEA, Patient's DEK encrypted with Doctor's public key)
- `is_active` (BOOLEAN, DEFAULT true)
- `shared_from` (TIMESTAMP WITH TIME ZONE)
- `shared_until` (TIMESTAMP WITH TIME ZONE, NULLABLE)

A.17 ResearchProjects Table

Defines research projects that users can consent to share data with.

- `research_project_id` (UUID, PK)
- `project_title` (VARCHAR)
- `principal_investigator_info_encrypted` (BYTEA)
- `institution_info_encrypted` (BYTEA)
- `irb_approval_id_encrypted` (BYTEA, NULLABLE)
- `project_description` (TEXT)
- `data_access_level_required` (VARCHAR, e.g., ANONYMIZED, PSEUDONYMIZED, IDENTIFIABLE)
- `status` (VARCHAR, e.g., ACTIVE, COMPLETED)
- `public_key` (TEXT, NULLABLE, Project's public key, potentially used if a model required researchers to decrypt a final dataset prepared by the SPE and encrypted specifically for the project (less common than SPE serving data via API, but allows for secure offline dataset delivery if absolutely necessary).)
- `created_at` (TIMESTAMP WITH TIME ZONE)

A.18 Researchers Table

Stores information about verified researchers who can access data via the Researcher Dashboard, based on project assignments and user consents.

- `researcher_id` (UUID, PK)
- `email` (VARCHAR, UNIQUE, Primary login identifier)
- `password_hash` (VARCHAR, Server-side hash of the researcher's password)
- `full_name_encrypted` (BYTEA, Client-side encrypted if considered PII, or plaintext if managed under organizational BAA/DPA)
- `institution_affiliation_encrypted` (BYTEA, NULLABLE)
- `verification_status` (VARCHAR, e.g., PENDING, VERIFIED, REJECTED)
- `role` (VARCHAR, NULLABLE, e.g., PRINCIPAL_INVESTIGATOR, RESEARCH_ASSISTANT)
- `created_at` (TIMESTAMP WITH TIME ZONE)
- `updated_at` (TIMESTAMP WITH TIME ZONE)

Note: The primary model for researcher access to identifiable or direct pseudonymized data is mediated by the SPE, which handles necessary decryption. A researcher-specific public key for direct DEK handling is generally not part of this core model to maintain stricter control and minimize DEK exposure.

A.19 ResearcherProjectAssignments Table (Junction)

Links researchers to the specific research projects they are authorized to work on. Access to data within these projects is further governed by user consents.

- `assignment_id` (UUID, PK)
- `researcher_id` (UUID, FK to `Researchers.researcher_id`)
- `research_project_id` (UUID, FK to `ResearchProjects.research_project_id`)
- `role_in_project` (VARCHAR, NULLABLE, e.g., LEAD, ANALYST, VIEWER)
- `access_granted_at` (TIMESTAMP WITH TIME ZONE)
- `access_revoked_at` (TIMESTAMP WITH TIME ZONE, NULLABLE)
- `is_active` (BOOLEAN, DEFAULT true)
- *Constraint: UNIQUE (researcher_id, research_project_id)*